Comprehensions with objects and iterators work, but they're a bit cumbersome. Suppose you don't have a keyboard event object and a dedicated iterator. Instead, you just have a function that produces a value:

```
UP = 1
DOWN = -1
keyboardEvents = ->
  if Math.floor Math.random()*10 > 5
    UP
  else
    DOWN
```

Wouldn't it be nice if there were some way to express a function that could work like an iterator on an object? You'd need some special syntax for a function that could give control back to the invoking function without being finished. There's such a thing proposed for ECMAScript 6 called a *generator function*. Instead of being evaluated and returning a value, a generator function *yields* values.

In order to support this new feature, ECMAScript 6 had to add new syntax to indicate that a function is a generator:

ECMAScript 6

```
const UP = 1;
const DOWN = -1;

function* keyboardEventGenerator () {
  for (;;) {
    if(Math.floor(Math.random()*10) > 5) {
      yield UP;
    }
    else {
      yield DOWN;
    }
  }
}
```

The new function* syntax indicates that the function being defined is a generator and should contain yield statements.

CoffeeScript will need to add new syntax to support the `yield` keyword for generators. One option is the starred arrow `->*`:

```
keyboardEventGenerator = ->*
  while true
    if Math.floor Math.random()*10 > 5
      yield UP
    else
      yield DOWN
```